Human-Powered Data Cleaning for Probabilistic Reachability Queries on Uncertain Graphs

Xin Lin, Yun Peng, Byron Choi, and Jianliang Xu

Abstract—Uncertain graph models are widely used in real-world applications such as knowledge graphs and social networks. To capture the uncertainty, each edge in an uncertain graph is associated with an existential probability that signifies the likelihood of the existence of the edge. One notable issue of querying uncertain graphs is that the results are sometimes uninformative because of the edge uncertainty. In this paper, we consider probabilistic reachability queries, which are one of the fundamental classes of graph queries. To make the results more informative, we adopt a crowdsourcing-based approach to clean the uncertain edges. However, considering the time and monetary cost of crowdsourcing, it is a problem to efficiently select a limited set of edges for cleaning that maximizes the quality improvement. We prove that the edge selection problem is #P-hard. In light of the hardness of the problem, we propose a series of edge selection algorithms, followed by a number of optimization techniques and pruning heuristics for reducing the computation time. Our experimental results demonstrate that our proposed techniques outperform a random selection by up to 27 times in terms of the result quality improvement and the brute-force solution by up to 60 times in terms of the elapsed time.

1 Introduction

Graph data arises in a large variety of real-world applications, such as social networks, knowledge graphs, and protein-protein interaction networks [2]. However, the data in these applications is often uncertain for various reasons that include unreliable data sources, ambiguity in the content of data, or approximate data models. For example, duplicate author names may confuse the coauthor networks that are automatically extracted from academic websites (e.g., DBLP and Google Scholar); in a machine-generated knowledge graph, the relationships between entities may be uncertain since they are usually extracted from web documents by using natural language processing techniques; in a protein-protein interaction network, the interactions between proteins are typically derived by a statistical model. To capture the uncertainty in these data, *uncertain graph model* has been a widely used data model, where each edge is associated with an existential probability that signifies the likelihood of the existence of the edge [14], [22].

In the literature, tremendous efforts on processing probabilistic queries on uncertain graphs have been reported. One of the most fundamental queries is reachability query [8], [14], [40], [41]. In a deterministic graph, a reachability query asks whether one vertex can reach another under some constraints, e.g., within a given distance or via a path that consists of a given set of labels. However, for an uncertain graph, the result of a reachability query is no longer a boolean value (i.e., "YES" or "NO"), but a probability of having the answer "YES". Existing works typically incorporate probabilities as follows. They assume the probabilities of the edges are independent of each other and adopt a *possible world model* to answer the queries. An uncertain graph may generate $O(2^m)$ possible worlds, where m is the number of the edges. Each possible world (or called *possible graph*) is a deterministic graph whose probability is the product of (i) the existential probabilities of all edges that exist in the graph and (ii) the non-existential probabilities of all edges that do not exist in the graph. The query result of an uncertain graph is the sum of the probabilities of the query results in all the possible worlds. Since the number of possible worlds is huge, the existing works focus on how to efficiently evaluate or estimate the query results.

A notable issue of querying uncertain graphs is that the results are sometimes uninformative, due to the uncertainty of edges. Consider the example in Fig. 1. If a user issues a query "Are vertices s and i reachable in two hops?", the probability of the answer "YES" is 0.5, derived by multiplying the probabilities of the two edges connecting them. (The probability of the answer "NO" is also 0.5.) The answer does not provide any meaningful information to the user. To remedy this issue, *data cleaning* is a widely adopted method to reduce the ambiguity of the query results, by reducing the uncertainty of some edges [19], [36]. In Fig. 1, if one uses some method to clean the edge (s, k) and finds its existential probability is 1 (or 0), then the query result of "YES" is 0.8 (or 0) which is more informative than before.

The emergence of crowdsourcing platforms naturally facilitates the cleaning operations described above. As discussed in [28], [36], humans outperform computers in many tasks like entity resolution and natural language processing. We argue that human-powered graph cleaning is helpful

- *X. Lin is with the Shanghai Key Laboratory of Multidimensional Information Processing, East China Normal University, Shanghai 200241, China, and the Department of Computer Science, Hong Kong Baptist University, Kowloon Tong, Hong Kong. E-mail: xlin@cs.ecnu.edu.cn.*
- *Y. Peng, B. Choi, and J. Xu are with the Department of Computer Science, Hong Kong Baptist University, Kowloon Tong, Hong Kong. E-mail: {ypeng, bchoi, xujl}@comp.hkbu.edu.hk.*

Manuscript received 7 Aug. 2016; revised 13 Feb. 2017; accepted 6 Mar. 2017. Date of publication 17 Mar. 2017; date of current version 1 June 2017. Recommended for acceptance by L. Dong.
For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.
Digital Object Identifier no. 10.1109/TKDE.2017.2684166

1041-4347 © 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

in a number of real-life scenarios. Below we list several examples:

- *Cleaning of Uncertain Social Networks.* A social network of academics can be extracted from publicly available bibliography repositories such as DBLP. Due to the duplicate and diverse forms of author names, the coauthor relationship may not be deterministic but probabilistic. People may want to use this coauthor relationship to discover some communities or to judge if two authors are involved in a two-hop relationship, for checking a potential conflict of interest. The probabilistic results could be uninformative, where data cleaning is needed. The probabilistic relationships involved can be posted to a crowdsourcing platform for cleaning. Humans in the crowd can simply resolve these relationships via external data sources, e.g., the homepages of the authors.
- *Cleaning of Uncertain Knowledge Graphs.* Some knowledge graphs are constructed by extracting public information from web data [12]. As stated in [12], due to the potentially dirty web data and the limitation of accuracy in natural language processing techniques, the relationships (e.g., couples and located at) between entities in a knowledge graph may be modeled by probabilistic edges. As such, the query results on the knowledge graph would be uncertain. The relationships between entities can also be posted on a crowdsourcing platform, in which the crowd uses the experience in real life or understands the corpus of the corresponding entities to clean them.

As discovered in [36], the best performance is obtained when crowdsourcing tasks are broken down into simple, small pieces. Consistent to this, the crowd usually performs better in cleaning a single probabilistic edge, when compared to directly reasoning ambiguous query results. For example, it is easier for the crowd to answer "Do Alice and Bob have a coauthor relationship?" than to answer "Do Alice and Bob have a two-hop coauthor relationship?". Therefore, in this paper, we undertake an edge cleaning approach to reduce the ambiguity and, hence, to promote the quality of query results.

Edge cleaning by crowdsourcing is costly in terms of time and monetary resources. The budget for edge cleaning on an uncertain graph might be limited. As such, we should carefully select the edges for crowdsourcing in order to maximize the quality improvement of query results. The main challenge of this problem is that it requires to aggregate the improved probabilities derived from numerous possible worlds to compute the quality improvement of query results. Meanwhile, the number of the candidates to-be-cleaned edges can be huge. We prove that the problem of selecting the optimal probabilistic edge for cleaning is #P-hard. To address the above challenges, we propose a general framework for edge selection. To improve the efficiency of edge selection, we deduce a factor which is correlated to the quality improvement so as to avoid costly probability computation. Furthermore, we devise an upper-bound based pruning technique to enable early termination of the edge selection procedure.

Our contributions made in this paper are summarized as follows:

- We identify the problem of data cleaning for reachability queries on uncertain graphs, which is proved to be #P-hard. To the best of our knowledge, this is the first study on this problem. We propose a general edge selection algorithm for cleaning the result of a reachability query.
- We extend the edge selection algorithm to promote the quality of multiple queries, which considers the joint cleaning effect among these queries.
- We also extend our algorithm to select multiple edges for cleaning in parallel. Since determining the exact optimal edge set for cleaning is very costly, we devise two heuristic algorithms to efficiently find the suboptimal edge set.
- We conduct extensive experiments to evaluate the performance of the proposed solutions. The results show that our proposed methods outperform a random selection by up to 27 times in terms of the result quality improvement. Meanwhile, in terms of the efficiency, our proposed methods are up to 60 times faster than the brute-force algorithm.

The rest of this paper is organized as follows. In Section 2, we present the background and related work. In Section 3, we define the problem and introduce some preliminaries about this problem. In Section 4, we focus on how to select the optimal edge to clean with respect to a single query. In Sections 5 and 6, the solution is respectively extended for selecting one edge for multiple queries and selecting multiple edges. Our experimental results are presented in Section 7. We conclude this paper in Section 8.

2 Related Work

2.1 Uncertain Graph Queries

Querying uncertain data has been a classical problem in database research [1]. Several innovative solutions, such as probabilistic threshold query [9], top-k query [24], and nearest neighbor query [21], have been proposed to solve the problem and its variants. One of the most widely adopted models for these works is the *possible world* model [3]. It can be briefly described as follows: Each uncertain object (or tuple) in a database is composed of several mutually exclusive instances, each of which is associated with a probability of its existence. An assumption is that all objects are independent. In each possible world, each object is assigned with the value of one instance. A possible world can then be regarded as a deterministic database. A possible world is also associated with a probability, which is simply the product of the probabilities of its instances. Queries on uncertain data are defined by combining the results of the queries on all possible worlds. However, listing all possible worlds is not feasible in many cases, since the number of possible worlds is exponential to the number of objects in the database. For this reason, the majority of existing works focuses on efficient techniques to evaluate queries on uncertain data, such as early stop [24] and approximation [14].

Uncertain graph databases have been emerging with various applications, e.g., social networks [37], knowledge graphs [12], biological networks [23]. There are also many

works on processing queries on uncertain graphs. Most of the works assume the edges of the graph are probabilistic. That is, there is a probability that an edge exists. Potamias et al. [22] propose a sampling-based algorithm for processing kNN queries on uncertain graphs. Yuan et al. [31] and Zou et al. [38] focus on two variants of shortest path queries, i.e., threshold-based shortest path query and top-k shortest path query. The studies presented in [18], [20], [33], [34] aim to evaluate pattern matching queries on uncertain graphs, while Yuan et al. [32] propose a solution to processing similarity search. Jin et al. [14] focus on evaluating distance-constraint reachability queries over uncertain graphs. They adopt a divide-and-conquer-based algorithm to determine an approximation of the reachability probability. Chen et al. [8] deal with label and distance-constraint reachability queries over uncertain graphs, which can be considered a variation of [32].

2.2 Data Cleaning for Uncertain Data

Data cleaning (or data cleansing) is a well-studied technique that aims to correct or remove inaccurate data from a database [10]. Specifically, for uncertain data, data cleaning is used to reduce the uncertainty in data. The most widely used model for uncertainty is the entropy-based model. In this model, the objective of the data cleaning algorithms is reducing the entropy of the whole result set, instead of promoting the probability of specific results. Chen et al. [7] focus on reducing the uncertainty of max and range query results on an uncertain database. Keulen et al. [15] use human's feedback to promote the quality of results from diverse databases. Knagal et al. [17] analyze the sensitivity and explanation of query results in probabilistic query results. Mo et al. [19] aim at reducing the uncertainty of top-k query results of an uncertain database.

The recent advances in the crowdsourcing platforms open up new opportunities for cleaning uncertain data. The crowd can help the system to reduce the inherent uncertainty in data, e.g., that produced by using artificial intelligence methods. Several existing works [11], [26], [27], [28], [29], [30] have proposed solutions to improve the quality of entity resolution. Zhang et al. study data cleaning for schema matching [36] and relational databases with a noisy crowd [35]. However, to the best of our knowledge, no existing work has investigated how to reduce uncertainty for reachability queries on uncertain graphs via crowdsourcing.

3 Preliminaries and Problem Definition

In this section, we first introduce some preliminaries, followed by the problem definition of this work. For better readability, we summarize the frequently used notations in Table 1. To ease presentation, we discuss our algorithms for undirected graphs only. The algorithms can be extended to handle directed graphs, since they do not make technical assumptions on directed or undirected graphs.

Definition 1 (Uncertain Graph). *Let $\mathcal{G} = (V, E, p, l)$ denote an* uncertain undirected graph, *where V and E are respectively the set of vertices and edges, $p : E \rightarrow (0, 1]$ denotes a function that maps an edge e to its probability of existence, and $l : E \rightarrow (0, \infty)$ maps an edge to its length (weight).*

TABLE 1
Frequently Used Notations

Notation	Meanings
$J_{s,t}$	The set of possible paths from s to t
$\mathcal{E}$	The set of candidate edges to be cleaned
$\mathcal{G}_c$	A new uncertain graph after applying the cleaning result c to $\mathcal{G}$
$R^d_{s,t}(\mathcal{G})$	The probabilistic reachability of s and t on the uncertain graph $\mathcal{G}$
$Q^d_{s,t}(\mathcal{G})$	The quality of the query result of $R^d_{s,t}(\mathcal{G})$
$E\Delta Q^d_{s,t}(\mathcal{G}, E_C)$	The excepted quality improvement of $R^d_{s,t}(\mathcal{G})$ on s and t after cleaning the set of edges in E_C
$Q_{ST}(\mathcal{G})$	The result quality of all queries in ST
$E\Delta Q_{ST}(\mathcal{G}, E_C)$	The expected quality improvement after cleaning edge set E_C
J_e	The d-path through edge e
$J_{\bar{e}}$	The d-path not through edge e
$l(e)$	The length of edge e
$dist(v_i, v_j)$	The shortest distance from v_i to v_j
$w(s, t)$	The weight of $\mathcal{PRQ}$ on vertices s and t
$Pr(j)$	The probability that all edges in path j exist
$G(J_1, J_2)$	The set of possible graphs in which all the paths in J_1 are present whereas all paths in J_2 are not.

Definition 2 (Possible Graph). *A possible graph $G = (V_G, E_G)$ of an uncertain graph $\mathcal{G}$, denoted by $G \subseteq \mathcal{G}$, is generated by sampling each edge e in $\mathcal{G}$ according to its existential probability $p(e)$.*

Slightly abusing the notations, in Definition 2, we use $G \subseteq \mathcal{G}$ to denote a possible graph G of $\mathcal{G}$. Specifically, E_G is a subset of edges E but the existence probabilities of edges have been made deterministic. According to the definition, there are $2^{|E|}$ possible graphs of $\mathcal{G}$. The *sampling probability* of G is calculated as follows:

$$Pr(G) = \prod_{e \in E_G} p(e) \cdot \prod_{e \notin E_G} (1 - p(e)). \tag{1}$$

Fig. 2 shows two exemplary possible graphs, G_1 and G_2, of the uncertain graph in Fig. 1. The sampling probability of G_1 is

$$\begin{aligned} Pr(G_1) &= p(s, f)p(s, a)p(g, a)p(g, h) \\ &\quad \cdot p(a, t)p(s, b)p(b, e)p(s, k)p(k, i) \\ &\quad \cdot (1 - p(g, f))(1 - p(h, t)) \\ &\quad \cdot (1 - p(b, t))(1 - p(e, t)) \\ &= 0.000016. \end{aligned}$$

A *path* from vertex v_0 to vertex v_n in $\mathcal{G}$ is a vertex sequence $(v_0, v_1, v_2, \ldots, v_i, \ldots, v_{n-1}, v_n)$, where (v_i, v_{i+1}) is an edge in $\mathcal{G}$. A *simple path* is a path such that there is no vertex appearing more than once. For simplicity, we use path and simple path interchangeably in the rest of the paper. A path is *present* in a possible graph G iff all edges in the sequence exist in E_G. The distance or length of a path is the sum of the lengths of all edges on the path. We denote by

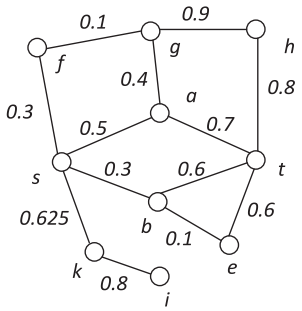


Fig. 1. An example of uncertain graph.

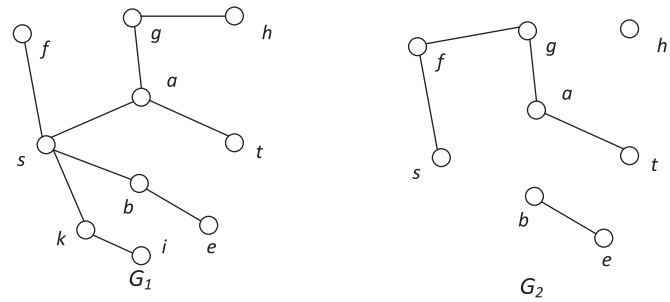


Fig. 2. An example of possible graphs.

$dist(s,t|G)$ the distance of the shortest path between two vertices s and t in a possible graph G.

Definition 3 (Deterministic Reachability). *Given two vertices s and t of a possible graph G and a distance threshold d, s and t are* reachable *in G iff $dist(s,t|G)$ is not longer than d.*

Let us take Fig. 2 for example. Assuming $d = 3$ and the length of each edge is 1, s and t are reachable in G_1 (via the path (s,a,t)), but not reachable in G_2.

Definition 4 (Probabilistic Reachability). *Given two vertices s and t of an uncertain graph $\mathcal{G}$ and a distance threshold d, the* probabilistic reachability *of s and t (denoted as $R^d_{s,t}(\mathcal{G})$) is the total probability of the possible graphs in which s and t are* reachable. *Formally, $R^d_{s,t}(\mathcal{G})$ is defined as follows:*

$$R^d_{s,t}(\mathcal{G}) = \sum_{G \subseteq \mathcal{G}} I^d_{s,t}(G) \cdot Pr(G), \tag{2}$$

where

$$I^d_{s,t}(G) = \begin{cases} 1, & \text{if } dist(s,t|G) \leq d \\ 0, & \text{otherwise.} \end{cases}$$

Definition 5 (Probabilistic Reachability Query). *Given an uncertain graph $\mathcal{G}$ and a distance threshold d, the probabilistic reachability query ($\mathcal{PRQ}$ for short) of s and t is to compute the value of $R^d_{s,t}(\mathcal{G})$.*

The range of $R^d_{s,t}(\mathcal{G})$ is evidently [0,1]. As mentioned in Section 1, if $R^d_{s,t}(\mathcal{G})$ is closer to 0 or 1, the quality of the $\mathcal{PRQ}$ result is higher since it can provide more deterministic information to the query issuer. Following the previous works [19], [36], we adopt information theory to measure the quality of the $\mathcal{PRQ}$ results.

Definition 6 (Quality of a $\mathcal{PRQ}$ Result). *Given two vertices s and t in an uncertain graph $\mathcal{G}$, the quality of a $\mathcal{PRQ}$ result is defined as follows:*

$$Q^d_{s,t}(\mathcal{G}) = h(R^d_{s,t}(\mathcal{G})) + h(1 - R^d_{s,t}(\mathcal{G})), \tag{3}$$

where $h(\cdot)$ is a function such that $h(x) = -x \log x$ and $h(0) = 0$.

Eq. (3) can also be explained as the entropy of the cases "YES" and "NO". According to the concept of entropy, a lower $Q^d_{s,t}(\mathcal{G})$ means lower uncertainty and, hence, higher quality.

We assume the cleaning of a probabilistic edge is to make its probability deterministic, that is, it is of the value either 1 or 0. Let E_C denote the set of edges to be cleaned and let $\mathcal{C}$ be the set of possible cleaning results. Clearly, the cardinality of $\mathcal{C}$ is $2^{|E_C|}$ since each edge in E_C can be cleaned to either 0 or 1.

The quality improvement due to a cleaning result $c_i \in C$ is then defined as follows:

$$\Delta Q^d_{s,t}(\mathcal{G}, c_i) = Q^d_{s,t}(\mathcal{G}) - Q^d_{s,t}(\mathcal{G}_{c_i}), \tag{4}$$

where $\mathcal{G}_{c_i}$ is the new uncertain graph after applying a cleaning result c_i to the original graph $\mathcal{G}$. Here, we assume the cleaning result of each edge is independent.

Since the cleaning is performed by the crowd, the cleaning results are not known beforehand. We use the *expected quality improvement* to quantify the benefit of a cleaning. The expected quality improvement can be formally expressed as follows:

$$E\Delta Q^d_{s,t}(\mathcal{G}, E_C) = Q^d_{s,t}(\mathcal{G}) - \sum_{c_i \in \mathcal{C}} Pr(\mathcal{G}_{c_i}) \cdot Q^d_{s,t}(\mathcal{G}_{c_i}). \tag{5}$$

According to the result of c_i, E_C can be divided into two disjoint sets I_{i0} and I_{i1}. For each edge e in I_{i0} (or I_{i1}), the returned result of e is 0 (or 1). Therefore, we have the following:

$$Pr(\mathcal{G}_{c_i}) = \prod_{e \in I_{i1}} p(e) \cdot \prod_{e \in I_{i0}} (1 - p(e)).$$

Example 1. Take Fig. 1 for example. If $d = 3$, there are three paths from s to t with a distance smaller than d, i.e., $j_1 : (s,a,t)$, $j_2 : (s,b,t)$ and $j_3 : (s,b,e,t)$. For a possible graph G, if any one of the above paths is present, $I^d_{s,t}(G) = 1$. Let $Pr(j)$ be the probability that the path j is present, which is computed by directly multiplying the existential probabilities of all its edges. We use $Pr(\bigvee_{j \in J} j)$ to denote the probability that at least one path in J is present and $Pr(\bigwedge_{j \in J} j)$ to denote the probability that all paths in J are present.

- $R^d_{s,t}(\mathcal{G}) = Pr(\bigvee_{j \in \{j_1,j_2,j_3\}} j) = Pr(j_1) + Pr(j_2) + Pr(j_3) - Pr(\bigwedge_{j \in \{j_1,j_2\}} j) - Pr(\bigwedge_{j \in \{j_1,j_3\}} j) - Pr(\bigwedge_{j \in \{j_2,j_3\}} j) + Pr(\bigwedge_{j \in \{j_1,j_2,j_3\}} j) = 0.35 + 0.18 + 0.018 - 0.35 * 0.18 - 0.35 * 0.018 - 0.6 * 0.6 * 0.3 * 0.1 + 0.6 * 0.6 * 0.3 * 0.1 * 0.35 = 0.472$. $Q^d_{s,t}(\mathcal{G}) = h(0.472) + h(1 - 0.472) = 0.692$.
- Assume that the edge to be cleaned is $e = (s,a)$. So $E_C = \{e\}$. $\mathcal{C}$ comprises two possible cleaning results: $\{c_1 : \{(e,1)\}, c_2 : \{(e,0)\}\}$. $\mathcal{G}_{c_1}$ and $\mathcal{G}_{c_2}$ are shown in Fig. 3.

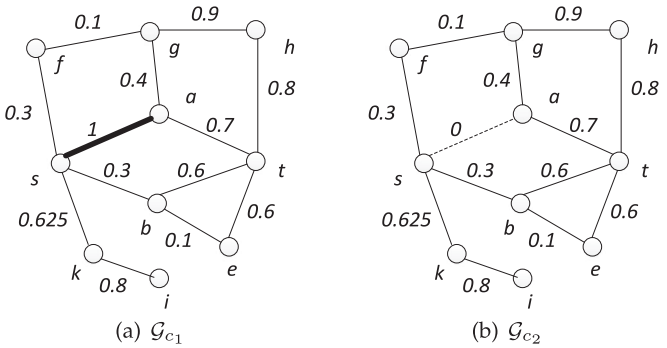


Fig. 3. An example of cleaned graph.

- $R^d_{s,t}(\mathcal{G}_{c_1}) = 0.756$, $R^d_{s,t}(\mathcal{G}_{c_2}) = 0.187$. Then, $Q^d_{s,t}(\mathcal{G}_{c_1}) = 0.555$, $Q^d_{s,t}(\mathcal{G}_{c_2}) = 0.482$. Since $Pr(\mathcal{G}_{c_1}) = Pr(\mathcal{G}_{c_2}) = p(s,a) = 0.5$, the expected quality improvement of the cleaning is $E\Delta Q^d_{s,t}(\mathcal{G}, \{(s,a)\}) = 0.692 - (0.5 * 0.555 + (1-0.5) * 0.482) = 0.220$.

Definition 7 (Quality of a Set of $\mathcal{PRQ}$ Results). *Given an uncertain graph $\mathcal{G}$, a set of vertex pairs $ST : V \times V$ and a distance distance d, the* quality of $\mathcal{PRQ}$ results *is*

$$Q_{ST}(\mathcal{G}) = \sum_{(s,t)\in ST} w(s,t) \times Q^d_{s,t}(\mathcal{G}), \tag{6}$$

where $w(s,t)$ is the weight associated with the $\mathcal{PRQ}$ on (s,t). The weights might be learned from the query logs, which represent the importance or query frequency of a $\mathcal{PRQ}$. By default, they are $\frac{1}{|ST|}$.

The expected quality improvement of the results of a set of $\mathcal{PRQ}$s is defined as follows:

$$E\Delta Q_{ST}(\mathcal{G}, E_C) = Q_{ST}(\mathcal{G}) - \sum_{c_i \in \mathcal{C}} Pr(\mathcal{G}_{c_i}) \times Q_{ST}(\mathcal{G}_{c_i}). \tag{7}$$

With the above definitions, we present *the problem statement* of this paper.

Definition 8 (Edge Cleaning Problem for $\mathcal{PRQ}$s). *Given an uncertain graph $\mathcal{G}$, a distance threshold d, a candidate edge set $\mathcal{E}$, and an edge cleaning budget n, the edge cleaning problem for $\mathcal{PRQ}$s is to find an edge set $E_C \subseteq \mathcal{E}$, $|E_C| = n$, that maximizes $E\Delta Q_{ST}(\mathcal{G}, E_C)$.*

Theorem 1. *The edge cleaning problem for $\mathcal{PRQ}$s is a #P-hard problem.*

The Sketch of the Proof. We prove the hardness of the Edge Cleaning Problem for $\mathcal{PRQ}$s (ECP) in two steps. First, we relate the hardness of an existing #P-complete problem to a specific problem that is close enough to ECP. Second, we establish the necessary tools for reducing the problem analyzed in the first part of the proof to a specific instance of ECP to establish the hardness of ECP. Due to the limitation of space, the full proof is presented in Appendix A in the supplementary file, which can be found on the Computer Society Digital Library at http://doi.ieeecomputersociety.org/10.1109/TKDE.2017.2684166. □

As such, it is necessary to study an efficient algorithm to practically solve the problem. In the following sections, we first assume $n = 1$ and propose the single-edge selection algorithm for a single $\mathcal{PRQ}$. Based on that, we then determine the single-edge that maximizes the quality improvement for multiple $\mathcal{PRQ}$s. Finally, we extend these techniques to the multiple-edge selection problem (where $n > 1$).

4 Single-Edge Selection Algorithm for Single Query

To propose a practical solution for the edge cleaning problem for a single $\mathcal{PRQ}$, we first deduce a factor that is correlated to $E\Delta Q$ for each edge, followed by discussing how to determine the edge with the maximum value of this factor.

4.1 Correlated Factor of Expected Quality Improvement

First, we introduce some notations for ease of presentation. Let $J_{s,t}$ denote the set of possible paths from s to t. We refer *d-path* to a subset of $J_{s,t}$ in which the distances of all paths are not larger than d, denoted as $J^d_{s,t}$. For an arbitrary edge e in the uncertain graph $\mathcal{G}$, the set $J^d_{s,t}$ can be further divided into two subsets: (i) J_e denotes the subset of paths in $J^d_{s,t}$ that pass through e, and (ii) $J_{\bar{e}}$ denotes the subset of paths in $J^d_{s,t}$ that do not go through e. Denote by $G(J_1, J_2)$ the set of possible graphs in which at least one path in J_1 is present whereas all paths in J_2 are not. We define $P^*(e) = \frac{\sum_{G\in G(J_e, J_{\bar{e}})} Pr(G)}{p(e)}$.[1]

Example 2. Let us revisit the example in Fig. 1, in which $d = 3$ and the length of each edge is 1. $J^d_{s,t}$ contains three paths $j_1 : (s,a,t)$, $j_2 : (s,b,t)$ and $j_3 : (s,b,e,t)$. Consider $e = (s,a)$. We have $J_e = \{j_1\}$, $J_{\bar{e}} = \{j_2, j_3\}$. $G(J_e, J_{\bar{e}})$ is the set of the possible graphs in which j_1 is present but is neither j_2 or j_3. Thus, $\sum_{G\in G(J_e,J_{\bar{e}})} Pr(G) = Pr(j_1) \times (1 - Pr(j_2 \bigvee j_3)) = 0.5 \times 0.7 \times (1 - (0.3 \times 0.6 + 0.3 \times 0.1 \times 0.6 - 0.3 \times 0.1 \times 0.6 \times 0.6)) = 0.285$. Since $p(e) = 0.5$, $P^*(e) = 0.570$.

The intuition of $P^*(e)$ is as follows. It is the total probability of the possible graphs that contain at least one path passing through e and do not contain any path not passing through e. If the probability of e is cleaned to 0, it will only influence those possible graphs. In other words, the deterministic reachability $I^d_{s,t}$ of those possible graphs will be changed from 1 to 0. On the other hand, if a possible graph contains some path(s) not passing through e, the cleaning result will not influence the $I^d_{s,t}$ value.

Theorem 2 proves that $P^*(e)$ is positively correlated to $E\Delta Q(\mathcal{G}, e)$.

Theorem 2. *Given two edges e_1 and e_2, $P^*(e_1) > P^*(e_2)$ implies that $E\Delta Q(\mathcal{G}, e_1) \geq E\Delta Q(\mathcal{G}, e_2)$.*

Proof. Let $EQ^d_{s,t}(\mathcal{G}, E_C) = \sum_{c\in\mathcal{C}} Pr(\mathcal{G}_c) \cdot Q^d_{s,t}(\mathcal{G}_c)$. So Eq. (5) can be transformed into

$$E\Delta Q^d_{s,t}(\mathcal{G}, E_C) = Q^d_{s,t}(\mathcal{G}) - EQ^d_{s,t}(\mathcal{G}, E_C). \tag{8}$$

□

Since $Q^d_{s,t}(\mathcal{G})$ is an invariant with respect to E_C, the edge selection problem is to minimize $EQ^d_{s,t}(\mathcal{G}, E_C)$. For the single-edge setting, we have

1. The subscript and superscript in $P^{d*}_{s,t}(e)$ are dropped for better readability.

$$EQ^d_{s,t}(\mathcal{G}, e) = p(e)Q^d_{s,t}(\mathcal{G}_{c_1}) + (1 - p(e))Q^d_{s,t}(\mathcal{G}_{c_2}), \quad (9)$$

where $\mathcal{G}_{c_1}$ (or $\mathcal{G}_{c_2}$) is the new uncertain graph in which the cleaning result of e is 1 (or 0).

According to Eq. (3), $Q^d_{s,t}(\mathcal{G}_{c_1})$ and $Q^d_{s,t}(\mathcal{G}_{c_2})$ can be further derived as follows:

$$Q^d_{s,t}(\mathcal{G}_{c_1}) = h(R^d_{s,t}(\mathcal{G}_{c_1})) + h(1 - R^d_{s,t}(\mathcal{G}_{c_1})) \quad (10)$$

$$Q^d_{s,t}(\mathcal{G}_{c_2}) = h(R^d_{s,t}(\mathcal{G}_{c_2})) + h(1 - R^d_{s,t}(\mathcal{G}_{c_2})). \quad (11)$$

Lemma 1 provides the derivation of $R^d_{s,t}(\mathcal{G}_{c_2})$ and $R^d_{s,t}(\mathcal{G}_{c_1})$.

Lemma 1. *$R^d_{s,t}(\mathcal{G}_{c_1}) = R^d_{s,t}(\mathcal{G}) + P^*(e) \cdot (1 - p(e))$, while $R^d_{s,t}(\mathcal{G}_{c_2}) = R^d_{s,t}(\mathcal{G}) - P^*(e) \cdot p(e)$.*

The Sketch of the Proof. The proof can be divided into two steps. The first one is to identify the possible graphs in which the reachability will be influenced by cleaning edge e. We prove that the total probability of such possible graphs is $P^*(e) \cdot p(e)$. The second step is to identify the reachability increasement (or reduction) by cleaning edge e in the above possible graphs. The lemma is deduced by multiplying these two parts. Due to the limitation of space, the full proof is presented in Appendix B in the supplementary file, available online. □

According to Eq. (10) and Lemma 1, we have

$$\frac{\mathrm{d}Q^d_{s,t}(\mathcal{G}_{c_1})}{\mathrm{d}P^*(e)} = \frac{\mathrm{d}R^d_{s,t}(\mathcal{G}_{c_1})}{\mathrm{d}P^*(e)} \cdot \frac{\mathrm{d}Q^d_{s,t}(\mathcal{G}_{c_1})}{\mathrm{d}R^d_{s,t}(\mathcal{G}_{c_1})} = (1 - p(e)) \cdot \log \frac{1 - R^d_{s,t}(\mathcal{G}_{c_1})}{R^d_{s,t}(\mathcal{G}_{c_1})}.$$

Similarly, we have

$$\frac{\mathrm{d}Q^d_{s,t}(\mathcal{G}_{c_2})}{\mathrm{d}P^*(e)} = -p(e) \cdot \log \frac{1 - R^d_{s,t}(\mathcal{G}_{c_2})}{R^d_{s,t}(\mathcal{G}_{c_2})}.$$

Substituting them into Eq. (9), we obtain

$$\begin{aligned}\frac{\mathrm{d}EQ^d_{s,t}(\mathcal{G}, e)}{\mathrm{d}P^*(e)} = &\frac{\mathrm{d}p(e)}{\mathrm{d}P^*(e)} \cdot Q^d_{s,t}(\mathcal{G}_{c_1}) \\ &+ p(e) \cdot \frac{\mathrm{d}Q^d_{s,t}(\mathcal{G}_{c_1})}{\mathrm{d}P^*(e)} \\ &+ \frac{\mathrm{d}(1 - p(e))}{\mathrm{d}P^*(e)} \cdot Q^d_{s,t}(\mathcal{G}_{c_2}) \\ &+ (1 - p(e)) \cdot \frac{\mathrm{d}Q^d_{s,t}(\mathcal{G}_{c_2})}{\mathrm{d}P^*(e)}.\end{aligned}$$

Since $\sum_{G \in G(J_e, J_{\bar{e}})} Pr(G)$ has a factor of $p(e)$, $P^*(e)$ and $p(e)$ are independent of each other. So, both $\frac{\mathrm{d}p(e)}{\mathrm{d}P^*(e)}$ and $\frac{\mathrm{d}(1-p(e))}{\mathrm{d}P^*(e)}$ are 0. As such, we have

$$\frac{\mathrm{d}EQ^d_{s,t}(\mathcal{G}, e)}{\mathrm{d}P^*(e)} = p(e) \cdot (1 - p(e)) \cdot \left[\log \frac{1 - R^d_{s,t}(\mathcal{G}_{c_1})}{R^d_{s,t}(\mathcal{G}_{c_1})} - \log \frac{1 - R^d_{s,t}(\mathcal{G}_{c_2})}{R^d_{s,t}(\mathcal{G}_{c_2})}\right].$$

Obviously, $R^d_{s,t}(\mathcal{G}_{c_1}) \geq R^d_{s,t}(\mathcal{G}_{c_2})$, $1 - R^d_{s,t}(\mathcal{G}_{c_1}) \leq R^d_{s,t}(\mathcal{G}_{c_2})$. Hence, $\log \frac{1-R^d_{s,t}(\mathcal{G}_{c_1})}{R^d_{s,t}(\mathcal{G}_{c_1})} - \log \frac{1-R^d_{s,t}(\mathcal{G}_{c_2})}{R^d_{s,t}(\mathcal{G}_{c_2})}$ must be no larger than 0. According to the definition of $p(e)$, we also have $p(e) \cdot (1 - p(e)) \geq 0$. As such, $\frac{\mathrm{d}EQ^d_{s,t}(\mathcal{G},e)}{\mathrm{d}P^*(e)} \leq 0$. Combining it with Eq. (8), $\frac{\mathrm{d}E\Delta Q(\mathcal{G},e)}{\mathrm{d}P^*(e)} \geq 0$ since $Q^d_{s,t}(\mathcal{G})$ is an invariant. Therefore, the theorem is proved.

4.2 Selecting the Edge with the Maximum $P^*(e)$

Owing to Theorem 2, our problem can be reduced to selecting the edge with the maximum $P^*(e)$ value. According to the definition of $P^*(e)$, it mainly consists of two factors, i.e., the denominator $p(e)$ and the numerator $\sum_{G \in G(J_e, J_{\bar{e}})} Pr(G)$. The straightforward method is computing the $P^*(e)$ values for all edges and selecting the best one. However, the derivation of the numerator of $P^*(e)$ is very inefficient since it should invoke many costly set union computation. Assuming the cardinalities of path set J_e and $J_{\bar{e}}$ are respectively $|J_e|$ and $|J_{\bar{e}}|$, the complexity of such set union computation is $O(2^{|J_e|} + 2^{|J_{\bar{e}}|})$. Hence, we propose an efficient algorithm that avoids computing all numerators.

Algorithm 1. Finding the Edge with the Maximum $P^*(e)$

INPUT: $\mathcal{G}$, s, t, d
OUTPUT: e_h: the edge with maximum $P^*(e)$
1: **for** each edge e in $\mathcal{E}$ **do**
2: compute the upper bound $U(e)$ of the numerator $\sum_{G \in G(J_e, J_{\bar{e}})} Pr(G)$
3: $NL \leftarrow$ the edge list sorted by $U(e)$ in descending order
4: $DL \leftarrow$ the edge list sorted by $p(e)$ in ascending order
5: $P^*_{max} \leftarrow -\infty$
6: $e_h \leftarrow$ *null*
7: **while** (*true*) **do**
8: $e_n \leftarrow NL$.pop()
9: $e_d \leftarrow DL$.pop()
10: **if** $\frac{U(e_n)}{p(e_d)} < P^*_{max}$ **then** //*computing the threshold*
11: **return** e^h
12: **if** $P^*(e_n) > P^*_{max}$ **then** //*calculating $P^*(e_n)$ by looking up $p(e_n)$ from DL*
13: $e_h \leftarrow e_n$
14: $P^*_{max} \leftarrow P^*(e_n)$
15: **if** $P^*(e_d) > P^*_{max}$ **then** //*calculating the denominator of $P^*(e_d)$ by Monte Carlo method*
16: $e_h \leftarrow e_d$
17: $P^*_{max} \leftarrow P^*(e_d)$

Algorithm 1 gives the overview of our algorithm. The main idea is similar to the TA algorithm for top-k computation [5]. We first derive the upper bound of the numerator for each edge in $\mathcal{E}$ (detailed later in this section). Then, we sort the upper bounds in descending order and sort the denominators in ascending order (Lines 3 and 4). After that, we sequentially access these two edge lists (Lines 7-17). In each iteration, after popping up the top edge of each list, we do random access to other sources to get the remaining parameters of the edge (e.g., connectivity probability, the paths going through it). For each accessed edge e, we adopt a *Monte-Carlo* method to compute the numerator of $P^*(e)$

(Lines 12 and 15). If $P^*(e)$ of an accessed edge e is larger than P^*_{max}, which records the maximal P^* value of previously accessed edges, P^*_{max} and e_h will be replaced by $P^*(e)$ and e (Lines 13-14 and 16-17). The terminating condition is that P^*_{max} is larger than $\frac{U(e_n)}{p(e_d)}$, where e_n and e_d are the immediate edges of two edge lists (Lines 10 and 11). Since $U(e_n)$ is the upper bound of $P^*(e_n)$'s numerator and NL is sorted by these upper bounds, $P^*(e)$ of the unaccessed edges will not exceed P^*_{max}.

In Algorithm 1, we need to compute the upper bound of the numerator of $P^*(e)$ for each edge as well as the exact values of $P^*(e)$ for a small number of (accessed) edges. In the following, we discuss: 1) how to exactly compute $P^*(e)$, especially for the numerator; 2) how to derive the upper bound of the numerator.

To compute the exact $P^*(e)$, one may first list all paths in $J^d_{s,t}$. We adopt a DFS-based solution to list them. After that, J_e and $J_{\overline{e}}$ can be easily identified. The numerator of $P^*(e)$ can be transformed as follows:

$$\sum_{G \in G(J_e, J_{\overline{e}})} Pr(G) = Pr\Big(\bigvee_{j \in J_e} j\Big) \cdot \Big[1 - Pr\Big(\bigvee_{j \in J_{\overline{e}}} j\Big)\Big], \tag{12}$$

where $Pr(\bigvee_{j \in J_e} j)$ means the probability of at least one path in the set J_e being present.

In [16], a *Monte-Carlo* method was introduced to estimate $Pr(\bigvee_{j \in J_e} j)$, which guarantees that the errors of the estimated values do not exceed a given range. However, the *Monte-Carlo* method may need to sample a large number of times to ensure this. Furthermore, we may call the Monte Carlo method once for each $P^*(e)$. As shown in [16], to achieve the (ε, δ)-precision, i.e., the probability that estimation error exceeds ε will not be larger than δ, the complexity is $O(\frac{|J_e|^2|E|}{\delta\varepsilon^2})$. Therefore, to reduce the computation cost, we use the upper bound of the numerator of $P^*(e)$ to generate the sorted list (Line 3 of Algorithm 1).

Based on Eq. (12), we can use the set-union principle to derive the upper bound of the numerator. According to Boole's inequality [4], we have

$$Pr\Big(\bigvee_{j \in J_e} j\Big) \leq \sum_{j \in J_e} Pr(j), \tag{13}$$

and

$$Pr\Big(\bigvee_{j \in J_e} j\Big) + Pr\Big(\bigvee_{j \in J_{\overline{e}}} j\Big) \geq Pr\Big(\bigvee_{j \in (J_e + J_{\overline{e}})} j\Big).$$

Since the probabilistic reachability from s to t within distance d can be transformed as $R^d_{s,t}(\mathcal{G}) = Pr(\bigvee_{j \in (J_e + J_{\overline{e}})} j)$, we get

$$Pr\Big(\bigvee_{j \in J_{\overline{e}}} j\Big) \geq R^d_{s,t}(\mathcal{G}) - Pr\Big(\bigvee_{j \in J_e} j\Big)$$

$$1 - Pr\Big(\bigvee_{j \in J_{\overline{e}}} j\Big) \leq \Big(1 - \Big(R^d_{s,t}(\mathcal{G}) - Pr\Big(\bigvee_{j \in J_e} j\Big)\Big)\Big). \tag{14}$$

Combining Eqs. (13) and (14), we have

$$\begin{aligned} & Pr\Big(\bigvee_{j \in J_e} j\Big) \cdot \Big[1 - Pr\Big(\bigvee_{j \in J_{\overline{e}}} j\Big)\Big] \\ & \leq Pr\Big(\bigvee_{j \in J_e} j\Big) \cdot \Big[1 - (R^d_{s,t}(\mathcal{G}) - Pr\Big(\bigvee_{j \in J_e} j\Big))\Big] \\ & \leq \sum_{j \in J_e} Pr(j) \cdot \Big[1 - (R^d_{s,t}(\mathcal{G}) - \sum_{j \in J_e} Pr(j))\Big]. \end{aligned}$$

Thus, we use $\sum_{j \in J_e} Pr(j) \cdot [1 - (R^d_{s,t}(\mathcal{G}) - \sum_{j \in J_e} Pr(j))]$ as the upper bound of the numerator, i.e., $U(e)$. Computing $U(e)$ for each e is light-weight here. Since we substitute the union operations with summation, for all $U(e)$'s, $R^d_{s,t}(\mathcal{G})$ is a constant value and we only need to call the *Monte-Carlo* method once to obtain $R^d_{s,t}(\mathcal{G})$. $Pr(j)$ can be directly obtained by multiplying the probabilities of all edges in the path j. For different e's, they can share the computation of $Pr(j)$ and $R^d_{s,t}(\mathcal{G})$ in the upper bound derivation.

4.3 Optimizations by Edge Pruning

In this section, we introduce two optimization techniques to prune the number of edges that take part in the computation of Algorithm 1. They are mainly based on two intuitions: 1) some edges are far away from s and t so that they have no chance to be a component of any d-path; 2) the $P^*(e)$ values of some edges are definitely smaller than those of the others.

Based on the above two intuitions, we propose two pruning rules as follows:

(1) *Pruning by reverse shortest paths.* The main idea can be divided into two steps: i) we first identify the vertices that will not be a vertex in any d-path; ii) all edges associated with these vertices are pruned. Specifically, we use Dijkstra's algorithm to compute the shortest distances between every vertex v to s and t, denoted by $ds(v)$ and $dt(v)$, respectively. If $ds(v) + dt(v) > d$, it means that there is no path with a distance smaller than d that passes through v. According to the definition of $P^*(e)$, the $P^*(e)$ value of any edge associated v must be 0 because $J_e = \phi$.

(2) *Pruning by edge dominance.* Assuming there are two edges e_1 and e_2, if both $p(e_1) < p(e_2)$ and $\sum_{G \in G(J_{e_1}, J_{\overline{e_1}})} Pr(G) \geq \sum_{G \in G(J_{e_2}, J_{\overline{e_2}})} Pr(G)$ hold, $P^*(e_1)$ is definitely larger than $P^*(e_2)$. In such a case, e_2 can be pruned. The comparison of the $P^*(e)$'s denominators is straightforward, while that of the numerator is more complex. One observation is if all paths through e_2 must pass through e_1, i.e., $J_{e_2} \subseteq J_{e_1}$, the numerator of $P^*(e_1)$ must be no smaller than that of $P^*(e_2)$. In this case, we call e_1 *dominates* e_2.

Fig. 4 shows the pruning result of the uncertain graph in Fig. 1. For pruning rule 1), we mark $(ds(v), dt(v))$ for each vertex v. For vertices f, g, h, k and i, $ds(\cdot) + dt(\cdot) > d = 3$. All edges associated with them can all be pruned (labeled by crosses). For pruning rule 2), all paths of connecting s and t through edge (b, t), e.g., j_2, must be through edge (s, b). Meanwhile since $p(s, b) < p(b, t)$, $p^*(s, b)$ must be

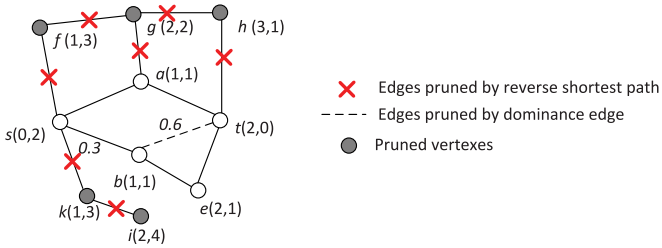


Fig. 4. An illustration of edge pruning.

larger than $p^*(b,t)$, which means the latter edge can be pruned.

Theorem 3 provides a sufficient condition to achieve the pruning by rule 2).

Theorem 3. *In the adjacent vertex set of v, if there is only one vertex v' such that $ds(v') < ds(v)$ (or $dt(v') < dt(v)$) and $p(v,v')$ is less than the probabilities of all other edges associated with v, it means these edges must be dominated by edge (v',v).*

Proof. The proof is straightforward. For vertex v, if there is only one adjacent vertex v' such that $ds(v') < ds(v)$ (or $dt(v') < dt(v)$), all paths passing through v must also pass through edge (v,v'). This means for each edge e associated with v except (v,v'), it holds that $J_e \subseteq J_{(v,v')}$ and $J_{\overline{(v,v')}} \subseteq J_{\overline{e}}$. Accordingly, the assertion $\sum_{G \in G(J_{(v,v')}, J_{\overline{e}})} Pr(G) \geq \sum_{G \in G(J_e, J_{\overline{(v,v')}})} Pr(G)$ holds. Combining with the condition $p(v,v') < p(e)$, the theorem follows. □

4.4 Optimizations for Large d Setting

Suppose that the distance threshold d is very large. It is not possible to list all paths from s to t since such listing is a #P-complete problem [25]. As such, we propose two heuristic algorithms to find an approximate solution.

- *Limited distance.* The first heuristic is based on the observation that the probabilities of the paths with a long distance are negligible, since the probability of a path is obtained by multiplying the probabilities of all its edges. Besides, the edges covered by shorter paths tend to be covered by longer paths as well. As such, we select the best edge under a smaller d setting as an approximation for a larger d, where the path listing for the smaller d is affordable in computation.
- *Limited probability.* The second heuristic is only considering the paths with a probability exceeding some threshold. The intuition is similar to the first heuristic, that is, the paths with a higher probability play a more important role in deciding the optimal edge. As such, we set a probability threshold τ. In the process of DFS-based path finding, if the probability of the current path is less than τ, the finding along this path can be terminated, which saves a lot of computation time.

5 Single-Edge Selection for Multiple Queries

Recall that our problem is, given a workload of multiple reachability queries, to find a number of edges for cleaning under a budget constraint. In this section, we extend our single-edge selection algorithm from singe query to multiple queries. We shall discuss the multiple-edge selection problem in Section 6.

The result quality concerned with multiple queries should consider the probabilistic reachabilities of all vertex pairs involved in the query workload. The number of vertex pairs is large for a graph with many queries. Nevertheless, for the cleaning of a given edge, it can only influence the probabilistic reachability of a limited set of vertex pairs. We call this set of vertex pairs the *influence set* of the edge. In the following, we first identify the influence set of each edge, followed by an efficient algorithm to find the edge that maximizes the quality improvement for multiple queries.

According to the analysis in Section 4.3, the cleaning of an edge will influence the $\mathcal{PRQ}$ result of a vertex pair only if the edge is on a d-path of the vertex pair. Assuming there is an edge (v_i, v_j) and a vertex pair s and t, the edge is on a d-path of s and t if and only if

$$dist(s, v_i) + l(v_i, v_j) + dist(v_j, t) \leq d, \tag{15}$$

or

$$dist(s, v_j) + l(v_i, v_j) + dist(v_i, t) \leq d, \tag{16}$$

where $dist(s,v)$ denotes the shortest distance from s to v in the uncertain graph $\mathcal{G}$ and $l(v_i, v_j)$ denotes the length of edge (v_i, v_j).

We derive the *influence set* of edge (v_i, v_j) as follows. First, we use Dijkstra's algorithm to compute the shortest distances between every vertex and v_i, v_j. For a pair of vertices s and t satisfying the inequalities (15) or (16), the pair (s,t) is put into the influence set of edge (v_i, v_j). In the case where d is small, the influence set is much smaller than the set covering all vertex pairs in $\mathcal{G}$, which would save the computation of selecting the best edge for cleaning.

A basic method to find the edge maximizing the quality improvement for multiple queries is to enumerate all edges and compare the new probabilistic reachabilities and expected quality improvement after cleaning them. However, as mentioned earlier, the reachability computation is essentially the set union of the paths, which is very costly. Even the *Monte-Carlo* method [16] may need to sample a large number of possible graphs to gain a high estimating accuracy. Hence, we should avoid as many reachability computation as possible. Similar to Algorithm 1, we use the upper bounds to help pruning. The details are shown in Algorithm 2. First, for each edge e, we compute the upper bound of the quality improvement for multiple queries by cleaning e, denoted by $UQ(e)$ (its derivation will be detailed in Section 5.1). After that, we sort these edges by $UQ(e)$ and access them sequentially. For each accessed edge e, we use the *Monte-Carlo* method to compute an estimated value of the expected quality improvement for multiple queries (EQ). We also maintain a variable EQ_{max} to record the largest EQ obtained so far. If the $UQ(e)$ value of the next edge is smaller than EQ_{max}, the algorithm is terminated and the edge associating with EQ_{max} is returned as the selection result. Since Algorithm 2 can be regarded as executing Algorithm 1 $|ST|$ times, where $|ST|$ is the number of queries, the time complexity of this Algorithm is $O(\frac{|ST||J_e|^2|E|}{\delta\varepsilon^2})$.

In the following section, we discuss how to derive $UQ(e)$ for a given edge e.

Algorithm 2. Single-Edge Selection for Multiple Queries

INPUT: $\mathcal{G}$, d, n
OUTPUT: e_h: the edge maximizing the quality improvement of the multiple queries

```
for each edge e in ℰ do
    IS(e) ← the influence set of e
    UQ(e) ← the upper bound of the quality improvement for multiple queries by cleaning e.
sort all edges e by the order of UQ(e) and store them in a max-heap ℋ
EQ_max ← −∞
while ℋ is not empty do
    e ← Dequeue(ℋ)
    if UQ(e) < EQ_max then
        return
    EQ ← 0
    for each vertex pair s and t in IS
        estimate EΔQ^d_{s,t}(𝒢), EΔQ^d_{s,t}(𝒢_{c1}) and EΔQ^d_{s,t}(𝒢_{c2}) by Monte Carlo method as in [16]
        EQ ← EQ + (EΔQ^d_{s,t}(𝒢) − p(e)EΔQ^d_{s,t}(𝒢_{c1}) - (1 − p(e)) EΔQ^d_{s,t}(𝒢_{c2}))
    if EQ > EQ_max then
        EQ_max ← EQ
        e_h ← e
return
```

5.1 Derivation of $UQ(e)$

Denote by ST the set of vertex pairs involved in the given query workload. According to Eq. (7),

$$E\Delta Q_{ST}(\mathcal{G}, e) = Q_{ST}(\mathcal{G}) - (p(e)Q_{ST}(\mathcal{G}_{c_1}) + (1 - p(e))Q_{ST}(\mathcal{G}_{c_2})).$$

Note that $Q_{ST}(\mathcal{G})$ is an invariant no matter which edge is selected for cleaning. Thus, deriving $UQ(e)$, the upper bound of quality improvement for multiple queries, is equivalent to deriving the lower bound of $p(e)Q_{ST}(\mathcal{G}_{c_1}) + (1 - p(e))Q_{ST}(\mathcal{G}_{c_2})$. For ease of presentation, we denote the above equation by $EQ_{ST}(e)$ and its lower bound by $E\check{Q}_{ST}(e)$. We have

$$UQ(e) = Q_{ST}(\mathcal{G}) - E\check{Q}_{ST}(e). \tag{17}$$

According to Eq. (6),

$$EQ_{ST}(e) = \sum_{(s_i,t_i)\in ST} w(s_i, t_i) \cdot (p(e)Q^d_{s_i,t_i}(\mathcal{G}_{c_1}) + (1 - p(e))Q^d_{s_i,t_i}(\mathcal{G}_{c_2})). \tag{18}$$

As defined above, $p(e)Q^d_{s_i,t_i}(\mathcal{G}_{c_1}) + (1 - p(e))Q^d_{s_i,t_i}(\mathcal{G}_{c_2}) = EQ^d_{s_i,t_i}(\mathcal{G}, e)$. We denote its lower bound by $E\check{Q}^d_{s_i,t_i}(\mathcal{G}, e)$. So we have

$$E\check{Q}_{ST}(e) = \sum_{(s_i,t_i)\in ST} w(s_i, t_i) \cdot E\check{Q}^d_{s_i,t_i}(\mathcal{G}, e). \tag{19}$$

The problem is reduced to deriving $E\check{Q}^d_{s_i,t_i}(\mathcal{G}, e)$ for each vertex pair (s_i, t_i). From the proof of Theorem 2, $EQ^d_{s_i,t_i}(e)$ is anticorrelated to $P^*(e)$ as s_i and t_i are the source and destination in $\mathcal{PRQ}$, which is denoted by $P^*_{s_i,t_i}(e)$. Thus, we can use the upper bound of $P^*_{s_i,t_i}(e)$ (denoted by $\hat{P}^*_{s_i,t_i}(e)$) to derive $E\check{Q}^d_{s_i,t_i}(\mathcal{G}, e)$.

Similar to the upper bound computation of $P^*(e)$ in Section 4.2, we can use $\sum_{j\in J_{e,s_i,t_i}} Pr(j) \cdot [1 - (R^d_{s,t}(\mathcal{G}) - \sum_{j\in J_{e,s_i,t_i}} Pr(j))]$ as $\hat{P}^*_{s_i,t_i}(e)$. According to Eqs. (9), (10), and (11), we have

$$\begin{aligned} E\check{Q}^d_{s_i,t_i}(\mathcal{G}, e) &= p(e)Q^d_{s_i,t_i}(\mathcal{G}_{c_1}) + (1 - p(e))Q^d_{s_i,t_i}(\mathcal{G}_{c_2}) \\ &= p(e)h(R^d_{s_i,t_i}(\mathcal{G}) + (1 - p(e))\hat{P}^*_{s_i,t_i}(e)) \\ &\quad + p(e)h(1 - R^d_{s_i,t_i}(\mathcal{G}) - (1 - p(e))\hat{P}^*_{s_i,t_i}(e)) \\ &\quad + (1 - p(e))h(R^d_{s_i,t_i}(\mathcal{G}) - p(e)\hat{P}^*_{s_i,t_i}(e)) \\ &\quad + (1 - p(e))h(1 - R^d_{s_i,t_i}(\mathcal{G}) + p(e)\hat{P}^*_{s_i,t_i}(e)). \end{aligned} \tag{20}$$

Substituting Eqs. (19) and (20) into Eq. (17), $UQ(e)$ can be derived.

6 Multiple-Edge Selection Algorithm for Multiple Queries

We now discuss how to select muliple edges for cleaning. We may invoke the single-edge selection algorithm repeatedly such that at each time an optimal edge is selected. Obviously, sequentially cleaning them is inefficient since the cleaning process in crowdsourcing takes time. An alternative is to select a batch of edges and clean them in parallel. Therefore, the multiple-edge selection problem is to find a set of edges $E_C \in \mathcal{E}$ which maximizes $E\Delta Q_{ST}(\mathcal{G}, E_C)$ as defined in Definition 7.

However, to exactly get the optimal result of E_C may need to enumerate all possible combinations of to-be-cleaned edges. The number of such combinations is $C^{|E_C|}_{|\mathcal{E}|}$. It is prohibitively costly when $|\mathcal{E}|$ and $|E_C|$ are large.

Inspired by the previous work on crowdsourcing-based cleaning, such as [19], [39], we propose two heuristic algorithms to optimize the performance.

- *Simple-extension.* The first one is a variant of the optimal polynomial single-edge selection algorithm proposed in Section 5. Assuming the budget of edge cleaning is n, we select the top-n edges with the highest EQ values in Algorithm 2. It can be easily achieved by slightly modifying the stop condition of this algorithm as to find n edges with EQ values larger than $UQ(e)$. Cleaning this set of edges is likely to yield a high joint quality improvement, but it may be suboptimal since it does not consider the mutual effect of the selected edges. Consider an uncertain graph containing four edges (s, a), (s, b), (t, a), (t, b), each with an existential probability of 0.7. Assume $n = 2$, $w(s, t) = 0.7$, and $w(a, b) = 0.3$. The $E\Delta Q_{ST}(\mathcal{G}, E_C)$ values of all edges are the same since they are all symmetric and we can randomly select two edges for cleaning according to this heuristic. However, the expected quality improvement of cleaning (s, a) and (a, t) will be higher than that of cleaning (s, a) and (s, b). It is because the former edge pair is involved in the same path, which augments the cleaning effect of each other.

- *Greedy*. The second heuristic greedily finds the to-be-cleaned edges in rounds, aiming to derive a local optimal solution for each round. More specifically, assuming we have selected a set of i to-be-cleaned edges E_C for an uncertain graph $\mathcal{G}$, we aim to find the $(i+1)$th edge e from the remaining edges such that $E\Delta Q_{ST}(\mathcal{G}, E_C + \{e\})$ is maximized. Each round of the edge selection is similar to Algorithm 2, in which we use the upper bounds to support pruning and avoid set-union computation. For each remaining edge e and each possible result combination c_i, we compute the upper bound of $E\Delta Q_{ST}(\mathcal{G}_{c_i}, e)$. After that, we combine these upper bounds by multiplying $Pr(\mathcal{G}_{c_i})$ to get $UQ(e)$ (Line 3 of Algorithm 2). Some $\mathcal{G}_{c_i}$ with little $Pr(\mathcal{G}_{c_i})$ is pruned here for saving computational time. Meanwhile, the computation of $EQ(e)$ for each edge e should also use $\mathcal{G}_{c_i}$ instead of $\mathcal{G}$ (Line 13 of Algorithm 2).

Denote the time complexity of the single-edge solution (Algorithm 2) by X, which is $O(\frac{|ST||J_e|^2|E|}{\delta\varepsilon^2})$ as stated in Section 5. The time complexity of simple-extension is also $O(X)$, while that of greedy is $O(nX)$.

7 Performance Evaluation

In this section, we conduct experimental studies to investigate the effectiveness and efficiency of our proposed algorithms.

7.1 Experiment Setup

Datasets. The following three datasets are used in performance evaluation:

- *ACD:* It is an uncertain coauthor graph constructed from the academic data published in the contest of KDD cup 2013.[2] The contest aimed to resolve the author-name ambiguity problem in Microsoft Academic Search platform. The ambiguity is caused by the variations of author names in different publications and the sharing of the same name among different authors. The authors were identified by joining their names and affiliations. We adopted the algorithm introduced in [13], which achieved an accuracy of 98 percent, to compute the linking probabilities between the authors and the papers in the original dataset. By these probabilities, we computed the coauthor probabilities between these authors and obtained an uncertain coauthor graph with 6,382 vertices and 23,179 edges.
- *NELL:* It is an uncertain costar graph consisting of 19,061 vertices and 108,538 edges. The "actorstarred-inmovie" relationships were extracted from the candidate briefs in the NELL knowledge base.[3] Each brief is associated with a probability. By these briefs, we computed the probabilities that two entities co-starred in a movie.
- *SYN:* To gain performance insights under various graphs, we randomly generated synthesized uncertain graphs by varying some parameters related to our algorithms, including the number of vertices, the average degree, and the distribution of the probabilities on the edges. The edges on the graph are randomly generated, and their existential probabilities follow a normal distribution with mean μ and variance σ^2. The corresponding parameters are summarized in Table 2.

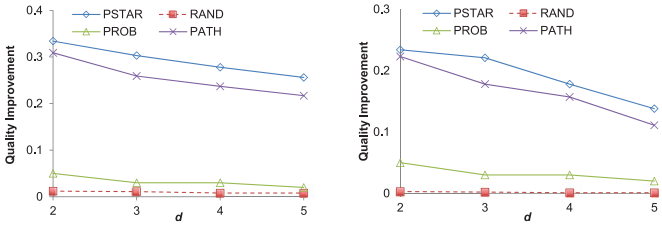
TABLE 2
Parameter Settings for Synthesized Datasets

Parameter	Value Range	Default
# Vertices	10,000 - 300,000	10,000
Average degree	2 - 16	4
σ^2	0.5-2	1
μ		0.5

Metrics and Implemented Algorithms. Our experiments aim to: (1) evaluate the effectiveness of our data cleaning algorithms for uncertain reachability queries; (2) evaluate the efficiency and scalability of our algorithms. As such, effectiveness and efficiency are two main metrics of the experiments:

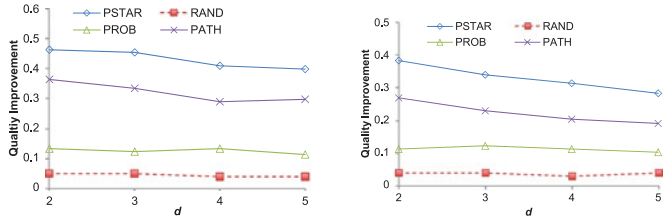
- *Effectiveness*. The effectiveness is measured by the quality improvement (Eq. (4)) owing to edge cleaning. The real datasets ACD and NELL are used to evaluate the effectiveness. The cleaning is achieved by posting the selected edges on Amazon Mechanical Turk[4] platform and getting the cleaning results from this platform. Each edge was assigned to five workers and each worker could get $0.01 as reward. The final cleaning result was obtained by adopting a majority vote on the answers from the workers. We compare our proposed algorithm (labeled by "PSTAR") with three straightforward algorithms. The first one (labeled by "RAND") randomly selects to-be-cleaned edges from the set of unpruned edges in Section 4.3. The second one (labeled by "PROB") follows the intuition that the edges with a probability close to 0.5 should be cleaned. Thus, we order the edges by $|0.5 - p|$ and sequentially select them to clean, where p is the existential probability of an edge. The third one (labeled by "PATH") considers the paths in which the edges are involved. For each edge, we sum up the probabilities of the paths in which it is involved and select the edges with the largest sums to clean.
- *Efficiency*. The efficiency measures the elapsed time of the edge selection process in each algorithm. Both the "RAND" and "PROB" algorithms can be finished within a very short time. However, the quality improvement gained by them are very low as will be shown in the effectiveness experiments. Hence, we will omit them in the efficiency experiments. For comparison, we also include two straightforward algorithms to study the effects of our optimizations. The first one exhausts all candidate edges and compares their expected quality improvements by Eq. (9) (labeled by "EXH"). The second one computes the

2. https://www.kaggle.com/c/kdd-cup-2013-author-paper-identification-challenge
3. http://rtw.ml.cmu.edu
4. www.mturk.com



(a) ACD (b) NELL

Fig. 5. Single-edge cleaning for single query.



(a) ACD (b) NELL

Fig. 6. Multiple edge cleaning for single query.

$P^*(e)$ values for all candidate edges but without using the upper bounds and sorting (labeled by "*PSTAR_N*"). For fairness, all algorithms adopt the two pruning methods introduced in Section 4.3 before edge selection. Also, all algorithms use the *Monte Carlo* method introduced in [16] to estimate the result of a set union operation.

For the multiple-edge and multiple-query experiments, all algorithms except "PATH" are extended by heuristic "Simple-extension", presented in Section 6, i.e., selecting the top-n edges with the highest expected quality improvements, where n is the budget of edge cleaning. The "PATH" algorithm selects the top-n edges with the largest aggregated path probabilities. We also implement the heuristic "Greedy" for "PSTAR", denoted by "PSTAR_G", for comparison.

Experiment Environment. We set the candidate to-be-cleaned edges, $\mathcal{E}$, as those with the original probabilities in the range [0.2, 0.8], since if the probability of an edge is close to 0 or 1, it is not very useful to clean it. The experiments were executed on a laptop (Intel Core i5 2.5 GHz CPU and 8 GB RAM) running Mac OS X 10.8.5 operating system. The codes were written in Java (JDK 1.6). Each experiment was repeated 100 times. All reported runtimes are the average of the measured results.

7.2 Performance of Single Query Setting

7.2.1 Effectiveness

Fig. 5 shows the results for the single query cleaning. We can see that the quality improvement gained by PSTAR is up to 0.33, which means the average probability of the query result can be improved from 0.5 to 0.85. It performs much better than RAND and PROB for both datasets ACD and NELL. Especially, the quality improvement of PSTAR is up to 27 times larger than RAND (for dataset ACD when $d = 2$) . PROB does not perform well as it only considers the probabilities of the edges, but not the topological relationships of the graph. The performance of all four algorithms deteriorates when the distance threshold d in the query increases. The reason is that as the value of d increases, the number of candidate edges increases and, hence, the effect of a single-edge cleaning degrades. Similarly, as the average degree in dataset NELL is larger than ACD, which produces more candidate paths and edges, the quality improvement for NELL is smaller than that for ACD.

Note that when d is set at 2, the performance of PATH is close to PSTAR. But as d increases, PATH degrades much faster than PSTAR. It is because PATH uses the summation of the path probabilities instead of the union. When d is 2, the summation is close to the union since the edges shared by different paths is very few. While d become larger, there will be more sharing edges, making the approximation less effective. Moreover, PSTAR assigns a deterministic value to the probability of the target edge, which makes the selection more precise.

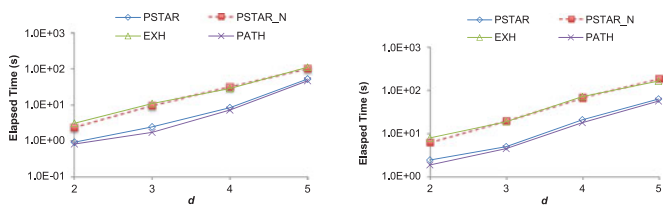
To further compare PSTAR with PATH, we also conducted a set of experiments which clean three edges on the graph. PSTAR adopts the heuristic "Simple-extension" in Section 6, while PATH selects the top-3 edges with the highest total probability of the involved paths. Fig. 6 shows that PSTAR achieves much higher joint quality improvement when compared to PATH.

7.2.2 Efficiency

We investigate the efficiency of the algorithms by varying d in the queries on both real datasets. As shown in Fig. 7, where the y-axis is on log-scale, the elapsed time of all algorithms degrades significantly as d grows. It is mainly because the relaxation of d increases the number of candidate paths and edges and hence prolongs the selection process. Nevertheless, we believe the performance is acceptable due to the following two reasons. First, in many real-world applications such as social networks, the d value used in reachability queries may not be very large. Second, many data cleaning solutions, such as crowdsourcing, are time-consuming so that the time cost spent in edge selection is neglectable. We can also see that our algorithm PSTAR is only slightly costly than PATH, since the major computation of these two algorithms is the same, i.e., listing all paths. On the other hand, thanks to the optimization techniques, PSTAR runs much faster than the other two algorithms EXH and PSTAR_N, since our algorithm prunes over 70 percent edges in the computation.

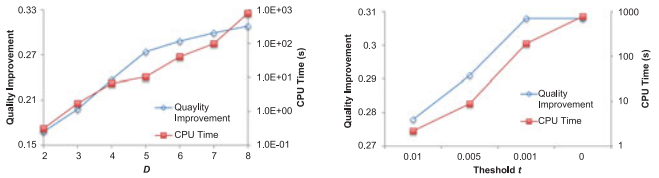
7.2.3 Experiment for Large d Setting

This set of experiments examines the effects of our heuristics proposed for large d settings. Since exact computing is infeasible for a large graph, we used a small synthetic graph for this set of experiments. The graph consists of 100 vertices



(a) ACD (b) NELL

Fig. 7. The effect of d.



(a) Effect of Limited Distance (b) Effect of Limited Probability

Fig. 8. Experiment for large d setting.

and the degrees of each vertex follow a power law distribution, which is observed in many large graphs, e.g., social networks. For heuristic "Limited distance", we assume that there are a series of reachability queries with a large d value. The value of d is set to 8 here. If d is over 8, the computation is so costly that we cannot obtain the performance of precisely selecting the best edges for comparison. Following the heuristic, we select to-be-cleaned edges by only considering the paths with a distance shorter than a parameter D. We compare the quality improvement by cleaning these edges. As shown in Fig. 8a, if we only consider the path with a length up to 5, the quality improvement by cleaning the selected edges can achieve 89 percent of the exact computing ($D = 8$). On the other hand, it saves up to 98.7 percent computation time comparing to the exact computing. For heuristic "Limited probability", we vary the threshold of the path probability. As shown in Fig. 8b, if we only consider the paths with a probability larger than 0.005, it achieves 94 percent quality improvement comparing to the exact computing, while it only uses 1 percent computation time. These results verify the effectiveness of the proposed heuristics.

7.3 Performance of Multiple-Query Setting

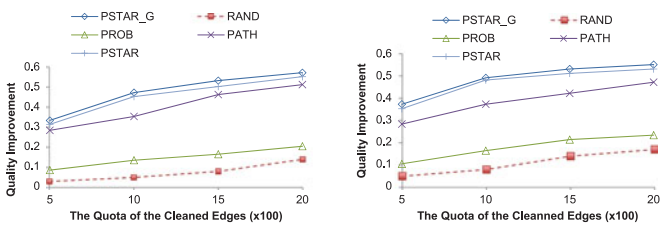
7.3.1 Effectiveness

We now evaluate the performance for multiple-query settings. We randomly select 2,000 reachability queries with an original probability in the range [0.2, 0.8]. The weights of these queries follow a normal distribution. We investigate the joint quality improvement with different budget quotas of edge cleaning.

As shown in Fig. 9, all algorithms gain more quality improvement with more budget quota. "PSTAR" and "PSTAR_G" perform better than the other algorithms for both datasets. "PSTAR_G" outperforms "PSTAR" since the former considers the mutual effect of the edges to be cleaned.

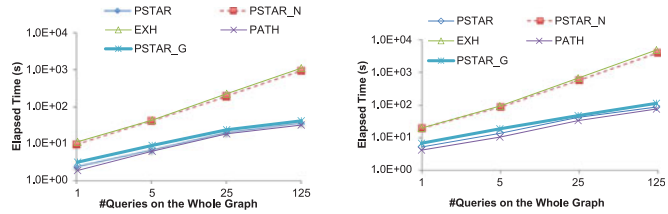
7.3.2 Efficiency

Fig. 10 shows the elapsed time of edge selection for multiple queries, where we vary the number of queries on the graph.



(a) ACD (b) NELL

Fig. 9. Multiple edge cleaning for multiple queries.



(a) ACD (b) NELL

Fig. 10. The effect of # queries.

As can be seen, the elapsed time is proportional to the number of queries. The reason is that the joint quality improvement for multiple queries is obtained by linearly combining those of the queries. Note that for dataset NELL, PSTAR performs 60 times faster than EXH when the number of queries is 125. And due to the same reasons stated in Section 7.2.2, the performance would be acceptable for data cleaning applications.

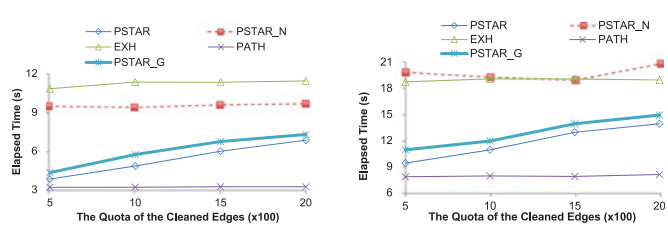
We also investigate the elapsed time of the algorithms by varying the budget quota for edge cleaning. As shown in Fig. 11, EHX, PSTAR_N, and PATH are not influenced by varying the quota, since they just enumerate all candidate edges and compute the corresponding values. PSTAR and PSTAR_G are degraded as the quota increases since the stop point is postponed. Nevertheless, their worst case does not exceed PSTAR_N as the latter does not use any sorting and optimization to accelerate the computing. Although PSTAR_G costs more elapsed time than PSTAR and PATH, it achieves a much higher quality improvement, which is more important for data cleaning applications.

7.4 Scalability and Stability

7.4.1 Effectiveness

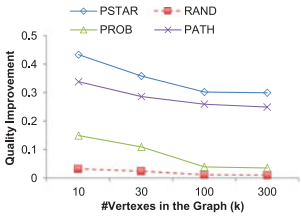
To test scalability, for each experiment we select 100 edges for cleaning in order to improve the quality of 100 randomly generated queries. The weights of these queries follow a normal distribution of mean 1 and variance 0.5.

We first investigate the performance by varying the graph size. As shown in the Fig. 12a, as the size of the graph grows, the performance of all the algorithms degrades. It is because in a smaller graph, the cleaning of one edge might be able to improve the quality of multiple queries. This effect degrades in a larger graph. We also vary the average degree on the synthesized graph. As shown in Fig. 12b, the quality improvement decreases significantly as the average degree increases. The reason is similar to that in Section 7.2.1: the higher degree produces more candidate paths and edges, which reduces the cleaning effect. Additionally, we vary the variance σ^2 to change the probability distribution of the edges. A larger variance means less

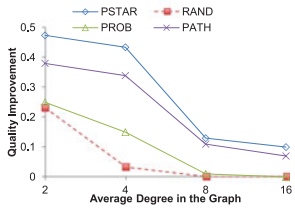


(a) ACD (b) NELL

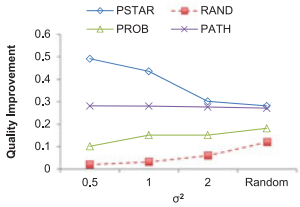
Fig. 11. The effect of edge cleaning quota.



(a) Effect of the Graph Size

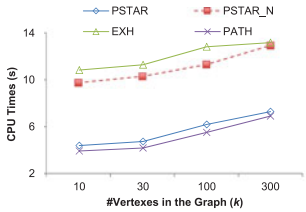


(b) Effect of the Average Degree

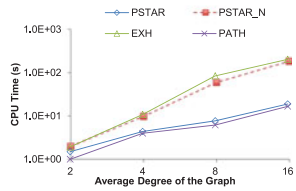


(c) Effect of the Probability Distribution

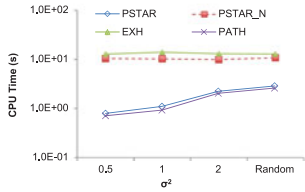
Fig. 12. The performance on the synthesized dataset.



(a) Effect of the Graph Size



(b) Effect of the Average Degree



(c) Effect of the Probability Distribution

Fig. 13. The performance on the synthesized dataset.

skewness. Fig. 12c shows that as σ^2 grows, the difference between PSTAR and PATH decreases. This implies our algorithm PSTAR performs better when the probabilities are more skewed. When the skewness decreases, the topological factor of the graph becomes dominant in the edge selection. In this case, the selections of PSTAR and PATH tend to be the same.

7.4.2 Efficiency

Finally, we investigate the efficiency of the algorithms on synthesized graphs. As shown in Fig. 13a, the increase of the graph size only slightly affects the efficiency of the algorithms. The extra elapsed time is due to more edge pruning (Section 4.3). Since a reachability query is only correlated to a set of local edges and vertices, the number of candidate edges is stable as the graph size grows up. Fig. 13b shows the effect of varying the average degree. The performance degrades significantly as the degree grows. The main reason is the increase of candidate paths and edges, which prolongs the computation of all the algorithms. Fig. 13c shows that the gap between PSTAR and the other algorithms become larger when the skewness increases, since skewness is helpful to early stop the computation. This again suggests our algorithm is more efficient when the probability distribution is more skewed.

8 Conclusion and Future Work

In this paper, we have studied the problem of data cleaning for probabilistic reachability queries on uncertain graphs. Due to the hardness of the problem, we proposed a correlated factor $P^*(e)$ to facilitate the edge selection in crowdscouring-based cleaning. We developed a number of optimization techniques and pruning heuristics for reducing the computation time in edge selection for both single queries and multiple queries. Extensive experimental results demonstrate the effectiveness and efficiency of our proposed algorithms under various system settings.

As for future work, we plan to extend our work to solve the problem in which the edges are correlated. The main idea is to redefine the $P^*(e)$ value such that the effects of both e and its corrected edges are considered. Moreover, we plan to use a more general weighted voting model to replace the majority voting model in aggregating the answers returned by the crowd. As the proposed edge selection algorithm can be extended to other decisional queries, we are also developing the details to support label-constraint reachability queries, cost-bounded reachability queries, and k-connectivity queries.

Acknowledgments

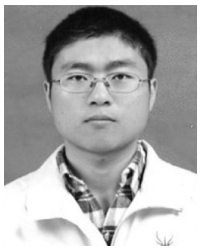
This work was supported by the National High-tech 863 Program of China (No. 2015AA015801) and HK-RGC grants 12244916, 12200114, and HKBU12201315. Xin Lin's work was also supported by NSFC (No. 61572193) and NSF of Shanghai (No. 17ZR1444900). Yun Peng's work was supported by NSFC (No. 61502258, 71402083) and NSF of Shandong (No. ZR2014FQ007). Yun Peng is the corresponding author.

References

[1] C. Aggarwal, *Managing and Mining Uncertain Data*. Berlin, Germany: Springer, 2009.
[2] R. Angles and C. Gutierrez, "Survey of graph database models," *ACM Comput. Surveys*, vol. 40, no. 1, pp. 1–39, 2008.
[3] C. Aggarwal and P. Yu, "A survey of uncertain data algorithms and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 5, pp. 609–623, May 2009.
[4] (2017). [Online]. Available: https://en.wikipedia.org/wiki/Boole's_inequality
[5] N. Bruno and H. Wang, "The threshold algorithm: From middleware systems to the relational engine," *IEEE Trans. Knowl. Data Eng.*, vol. 19, no. 4, pp. 523–537, Apr. 2007.
[6] C. Colbourn, "Analysis and synthesis problems for pairwise reachability," *Math. Comput. Model.*, vol. 17, no. 11, pp. 43–48, 1993.
[7] R. Cheng, J. Chen, and X. Xie, "Cleaning uncertain data with quality guarantees," *Proc. VLDB Endowment*, vol. 1, pp. 722–735, 2008.
[8] M. Chen, Y. Gu, Y. Bao, and G. Yu, "Label and distance-constraint reachability queries in uncertain graphs," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, 2014, pp. 188–202.
[9] R. Cheng, Y. Xia, S. Prabhakar, R. Shah, and J. Vitter, "Efficient indexing methods for probabilistic threshold queries over uncertain data," in *Proc. 30th Int. Conf. Very Large Data Bases*, 2004, pp. 876–887.
[10] (2017). [Online]. Available: https://en.wikipedia.org/wiki/Data_cleansing
[11] G. Demartini, D. Difallah, and P. Cudr é-Mauroux, "ZenCrowd: Leveraging probabilistic reasoning and crowdsourcing techniques for large-scale entity linking," in *Proc. 21st Int. Conf. World Wide Web*, 2012, pp. 469–478.

[12] X. Dong, et al., "Knowledge vault: A Web-scale approach to probabilistic knowledge fusion," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2014, pp. 601–610.
[13] D. Efimov, L. Silva, and B. Solecki, "KDD Cup 2013-Author-paper identification challenge: Second place team," in *Proc. KDD Cup Workshop*, 2013, Art. no. 3.
[14] R. Jin, L. Liu, B. Ding, and H. Wang, "Distance-constraint reachability computation in uncertain graphs," *Proc. VLDB Endowment*, vol. 4, pp. 551–562, 2011.
[15] M. van Keulen and A. de Keijzer, "Qualitative effects of knowledge rules and user feedback in probabilistic data integration," *VLDB J.*, vol. 18, no. 5, pp. 1191–1217, 2009.
[16] R. Karp and M. Luby, "A new Monte-Carlo method for estimating the failure probability of an n-component system," Univ. California, Berkeley, Berkeley, CA, USA, Tech. Rep. UCB/CSD-83-117, 1983.
[17] B. Kanagal, J. Li, and A. Deshpande, "Sensitivity analysis and explanations for robust query evaluation in probabilistic databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2011, pp. 841–852.
[18] X. Lian and L. Chen, "Efficient query answering in probabilistic RDF graphs," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2011, pp. 157–168.
[19] L. Mo, R. Cheng, X. Li, D. W. Cheung, and X. Yang, "Cleaning uncertain data for top-k queries," in *Proc. IEEE 29th Int. Conf. Data Eng.*, 2013, pp. 134–145.
[20] W. Moustafa, A. Kimmig, A. Deshpande, and L. Getoor, "Subgraph pattern matching over uncertain graphs with identity linkage uncertainty," in *Proc. IEEE 30th Int. Conf. Data Eng.*, 2014, pp. 904–915.
[21] J. Niedermayer, et al., "Probabilistic nearest neighbor queries on uncertain moving object trajectories," *Proc. VLDB Endowment*, vol. 7, pp. 205–216, 2013.
[22] M. Potamias, F. Bonchi, A. Gionis, and G. Kollios, "k-nearest neighbors in uncertain graphs," *Proc. VLDB Endowment*, vol. 3, pp. 997–1008, 2010.
[23] P. Sevon, L. Eronen, P. Hintsanen, K. Kulovesi, and H. Toivonen, "Link discovery in graphs derived from biological databases," in *Proc. 3rd Int. Conf. Data Integr. Life Sci.*, 2006, pp. 35–49.
[24] M. Soliman, I. Ilyas, and K. Chang, "Top k-query processing in uncertain databases," in *Proc. IEEE 23rd Int. Conf. Data Eng.*, 2007, pp. 896–905.
[25] L. Valiant, "The complexity of enumeration and reliability problems," *SIAM J. Comput.*, vol. 8, no. 3, pp. 410–421, 1979.
[26] N. Vesdapunt, K. Bellare, and N. Dalvi, "Crowdsourcing algorithms for entity resolution," *Proc. VLDB Endowment*, vol. 7, pp. 1071–1082, 2014.
[27] V. Verroios and H. Garcia-Molina , "Entity resolution with crowd errors," in *Proc. IEEE 31st Int. Conf. Data Eng.*, 2015, pp. 219–230.
[28] J. Wang, G. Li, T. Kraska, M. Franklin, and J. Feng, "Leveraging transitive relations for crowdsourced joins," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2013, pp. 229–240.
[29] S. Whang, P. Lofgren, and H. Garcia-Molina , "Question selection for crowd entity resolution," *Proc. VLDB Endowment*, vol. 6, pp. 349–360, 2013.
[30] J. Wang, T. Kraska, M. Franklin, and J. Feng, "CrowdER: Crowdsourcing entity resolution," *Proc. VLDB Endowment*, vol. 5, pp. 1483–1494, 2012.
[31] Y. Yuan, L. Chen, and G. Wang, "Efficiently answering probability threshold-based shortest path queries over uncertain graphs," in *Proc. 15th Int. Conf. Database Syst. Adv. Appl.*, 2010, pp. 155–170.
[32] Y. Yuan, G. Wang, L. Chen, and H. Wang, "Efficient subgraph similarity search on large probabilistic graph databases," *Proc. VLDB Endowment*, vol. 5, pp. 800–811, 2012.
[33] Y. Yuan, G. Wang, and L. Chen, "Pattern match query in a large uncertain graph," in *Proc. 23rd ACM Int. Conf. Conf. Inf. Knowl. Manage.*, 2014, pp. 519–528.
[34] Y. Yuan, G. Wang, H. Wang, and L. Chen, "Efficient subgraph search over large uncertain graphs," *Proc. VLDB Endowment*, vol. 4, pp. 876–886, 2011.
[35] C. Zhang, L. Chen, Y. Tong, and Z. Liu, "Cleaning uncertain data with a noisy crowd," in *Proc. IEEE 31st Int. Conf. Data Eng.*, 2015, pp. 6–17.
[36] C. Zhang, L. Chen, H. Jagadish, and C. Cao, "Reducing uncertainty of schema matching via crowdsourcing," *Proc. VLDB Endowment*, vol. 6, pp. 757–768, 2013.
[37] Z. Zou, H. Gao, and J. Li, "Discovering frequent subgraphs over uncertain graph databases under probabilistic semantics," in *Proc. 16th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2010, pp. 633–642.
[38] L. Zou, P. Peng, and D. Zhao, "Top-k possible shortest path query over a large uncertain graph," in *Proc. 12th Int. Conf. Web Inf. Syst. Eng.*, 2011, pp. 72–86.
[39] E. Ciceri, P. Fraternali, D. Martinenghi, and M. Tagliasacchi, "Crowdsourcing for top-k query processing over uncertain data," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 1, pp. 41–53, Jan. 2016.
[40] J. Jiang, P. Yi, B. Choi and X. Yu, "Privacy-preserving reachability query services for massive networks," in *Proc. 25th ACM Int. Conf. Inf. Knowl. Manage.*, 2016, pp. 145–154.
[41] J. Cheng, S. Huang, H. Wu, and A. Fu, "TF-label: A topological-folding labeling scheme for reachability querying in a large graph," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2013, pp. 193–204.

Xin Lin received the BEng and PhD degrees, both in computer science and engineering, from Zhejiang University, China. He is currently an associate professor in the Department of Computer Science, East China Normal University. He is also a visiting scholar in the Database Group, Hong Kong Baptist University (http://www.comp.hkbu.edu.hk/∼db/). His research interests include location-based services, spatial databases, privacy-aware computing, and crowdsourcing.



Yun Peng received the BSc degree in computer science from Shandong University, in 2006, the MPhil degree in computer science from the Harbin Institute of Technology, in 2008, and the PhD degree in computer science from Hong Kong Baptist University, in 2013. His research interests include graph data processing, data outsourcing, view update, and incremental maintenance.

Byron Choi received the BE degree in computer engineering from the Hong Kong University of Science and Technology (HKUST), in 1999, and the MSE and PhD degrees in computer and information science from the University of Pennsylvania, in 2002 and 2006, respectively. He is an associate professor in the Department of Computer Science, Hong Kong Baptist University.

Jianliang Xu is a professor in the Department of Computer Science, Hong Kong Baptist University (HKBU). He held visiting positions with Pennsylvania State University and Fudan University. His current research interests include data management, database security & privacy, and location-aware computing. He has published more than 150 technical papers in these areas, most of which appeared in leading journals and conferences including SIGMOD, VLDB, ICDE, the *ACM Transactions on Database Systems*, the *IEEE Transactions on Knowledge and Data Engineering*, and the *Very Large Data Base Journal*. He has served as a program co-chair/vice chair for a number of major international conferences including IEEE ICDCS 2012, IEEE CPSNA 2015, and WAIM 2016. He is an associate editor of the *IEEE Transactions on Knowledge and Data Engineering* and the *Proceedings of the VLDB Endowment* (*PVLDB* 2018).

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.